ĐỀ CƯƠNG CHI TIẾT VÀ ĐẶC TẢ PHẠM VI — NHÓM 18

Môn: Hệ thống và Mạng máy tính (MI4060)
Chủ đề lớn: Topic 1 — AI-Driven Data Center Fabrics
Tên đề tài: Nghiên cứu và xây dựng nguyên mẫu (prototype) giám sát, đánh giá cấu hình mạng Data Center Spine–Leaf cho lưu lượng AI Inference
Quy mô thực hiện: 4 thành viên · 12 tuần · chi phí phần mềm/hạ tầng dự kiến 0 đồng khi tận dụng máy cá nhân.
Tính chất tài liệu: Sườn báo cáo và đặc tả quản lý phạm vi; chưa phải kết quả nghiên cứu, không bao gồm dữ liệu thực nghiệm đã thu được.

> **Phạm vi đã chốt:** Prototype testbed mạng Spine–Leaf + monitoring/diagnosis + cảnh báo + khuyến nghị dựa trên quy tắc + người vận hành xác nhận Apply trên testbed + đo lại và so sánh cấu hình. **Không** tự phục hồi, không điều khiển mạng production và không chạy LLM/GPU thật.

A. Tóm tắt bài toán và sản phẩm

A.1. Bối cảnh, người sử dụng và vấn đề

Bối cảnh giả định: một doanh nghiệp dự kiến triển khai dịch vụ AI inference (chatbot/trợ lý nội bộ/hệ thống hỏi đáp) trong Data Center. Người sử dụng **dự kiến** của prototype là kỹ sư Hạ tầng CNTT/Network Operations phối hợp với bộ phận AI Platform. Họ cần kiểm tra các cấu hình mạng trước khi áp dụng vào môi trường thật. Đây là **bối cảnh minh họa**, không phải tuyên bố nhóm đã khảo sát hay có dữ liệu nội bộ của một doanh nghiệp cụ thể.

Khi nhiều yêu cầu AI inference truyền đồng thời, đường truyền và hàng đợi trên đường client–server có thể bão hòa; RTT tăng, throughput từng luồng giảm, đôi khi có packet loss. Đo từng metric riêng lẻ không đủ xác định *tắc ở link nào* và *phương án cấu hình nào cải thiện hiệu năng*. Thử trực tiếp nhiều cấu hình trên mạng doanh nghiệp tiềm ẩn rủi ro gián đoạn, còn mô tả lý thuyết không tạo được bằng chứng định lượng.

Vấn đề cần giải quyết: thiếu một môi trường kiểm thử **tái lập được**, có thể sinh tải đại diện AI inference, quan sát nghẽn, hỗ trợ thử cấu hình mạng khác và chứng minh tác động bằng số liệu trước–sau.

A.2. Câu hỏi nghiên cứu (cần xuất hiện nguyên mục trong báo cáo)

RQ chính: Với workload AI inference được xác định và kiểm soát, kiến trúc Spine–Leaf có hiệu năng như thế nào dưới các mức tải khác nhau, và việc thử cấu hình khác có cải thiện được các chỉ số mạng hay không?

- **RQ1 — Tải và hiệu năng:** khi tăng số luồng đồng thời 1/2/4/8, RTT và throughput (tổng/từng luồng) thay đổi ra sao? Mức tải nào bắt đầu có dấu hiệu bão hòa trong **môi trường mô phỏng đã cấu hình**?
- **RQ2 — Chẩn đoán:** sử dụng link utilization kết hợp throughput và RTT có xác định đúng link chịu tải tới hạn không? Bằng chứng kiểm chứng đường đi và counter là gì?

- **RQ3 — Lựa chọn cấu hình:** với cùng workload và điều kiện đo, cấu hình A (single-path) và B (hai đường/flow-splitting hoặc ECMP nếu triển khai đúng kỹ thuật) khác nhau như thế nào? Sau khi người vận hành chọn Apply, kết quả có được ghi nhận và so sánh lại đúng cách không?

Giả thuyết cần kiểm chứng, không được viết thành kết luận có sẵn: ở cấu hình có chia tải hợp lệ, một link quá tải có thể giảm tải nếu đường dự phòng thực sự còn dung lượng; hiệu quả không được mặc định là luôn tích cực.

A.3. Giải pháp/sản phẩm cuối cùng

Sản phẩm chính: một **prototype giám sát và hỗ trợ đánh giá cấu hình mạng Spine–Leaf trên testbed emulation**, có thể demo trọn vòng:

`Tạo tải → Monitor → Detect → Recommend → Người vận hành xác nhận → Apply trên testbed → Verify → Compare`.

Hai lớp chính: 1. **Network Testbed:** Mininet/Open vSwitch, 10 node logic, mô phỏng luồng request/response giữa client và server emulator. 2. **Monitoring & Configuration Support:** script thu thập metric, luật phát hiện bottleneck, cảnh báo, gợi ý cấu hình, thao tác Apply/Ignore và báo cáo so sánh.

Phân biệt: đây **không** phải Network Management System điều khiển switch thật, cũng không phải self-healing. Thay đổi chỉ có hiệu lực trong testbed, sau **xác nhận của người vận hành**.

A.4. Đầu vào → xử lý → đầu ra

Thành phần	Đặc tả đã chốt	Lưu ý
Đầu vào cấu hình mạng	Topology, link bandwidth/delay, IP/routing/firewall, profile A/B	Lưu cấu hình và version từng lần chạy
Đầu vào workload	Client/server, request size, response size, request rate, server service delay, số flow	Giữ các biến khác cố định khi tăng số flow
Đầu vào giám sát	Chu kỳ lấy mẫu, baseline, ngưỡng cảnh báo và thời gian duy trì ngưỡng	Không kết luận bottleneck từ một điểm đo
Đầu vào tương tác	Operator chọn **Apply/Ignore**	Đây là hành động người dùng, không phải thông số workload
Xử lý	Dựng mạng, sinh tải, thu metric, detect, recommend, apply (có xác nhận), rerun, compare	Quy tắc minh bạch, có log xác minh

Thành phần	Đặc tả đã chốt	Lưu ý
Đầu ra vận hành	RTT, throughput, d_trans (lý thuyết), utilization, bottleneck, trạng thái/cảnh báo, khuyến nghị, log Apply	Không gọi RTT là thời gian suy luận mô hình
Đầu ra quyết định	Bảng/biểu đồ A–B, kết luận *trong các cấu hình đã thử*	Không suy rộng sang mọi Data Center
Sản phẩm bàn giao	Code, cấu hình, scripts, giao diện, raw logs/CSV, biểu đồ, README, báo cáo, slides/demo	AI Prompts & Verification Log nếu dùng AI hỗ trợ code

A.5. Mức giới hạn và tiêu chí chốt phạm vi

- **Môi trường:** máy Windows/macOS của thành viên chạy Ubuntu VM thích hợp với kiến trúc máy; cùng phiên bản hệ điều hành/phần mềm và cùng script thiết lập. Không yêu cầu dùng chung một image nhị phân trên x86 và Apple Silicon.
- **Topology cơ sở:** 2 client + 2 AI server emulator + 2 leaf + 2 spine + 1 gateway/router + 1 firewall = **10 node logic**; gateway/firewall phải không thành bottleneck ngoài ý muốn của phép đo fabric.
- **Luồng:** 2 client logic có thể tạo tổng 1/2/4/8 kết nối đồng thời; không cần 8 máy tính/host riêng.
- **Workload:** request/response **giả lập** có tham số rõ ràng, không tuyên bố là dữ liệu traffic doanh nghiệp đã đo. Không cần LLM/GPU/API trả phí.
- **Cấu hình cần so sánh (P0):** A = single-path và B = chuyển/phân chia một số flow có kiểm soát qua hai đường spine. Chỉ gọi B là **ECMP** khi cơ chế hash/next-hop multipath được triển khai và chứng minh đúng; nếu dùng OpenFlow tĩnh phân luồng thì phải ghi đúng tên là **flow-splitting**.
- **Phát hiện:** kết hợp counter utilization cao duy trì liên tục với dấu hiệu throughput bão hòa và/hoặc RTT tăng so với baseline; rule/threshold công bố được, có trường hợp kiểm thử âm tính và dương tính.
- **Áp dụng:** chỉ profile định trước và được kiểm thử; hệ thống yêu cầu người vận hành xác nhận, ghi log cấu hình cũ/mới, kiểm tra lại đường đi sau Apply. **Không tự sửa sản xuất, không tự recovery khi link hỏng.**
- **Phương pháp:** emulation + đối chứng A/B có kiểm soát + lặp tối thiểu 3 lần/testcase; thống nhất các biến độc lập. Các phép đo chạy trên VM có hạn chế về sai lệch timing/CPU.
- **Ngoài phạm vi:** GPU cluster/LLM thật, training, model-parallel, NCCL/RDMA/InfiniBand, Kubernetes/multi-cloud, RAG, AI-based routing, anomaly/self-healing, network production, khảo sát dữ liệu riêng của doanh nghiệp.

B. Đề cương báo cáo theo feedback của giảng viên

> **Cấu trúc cố định: 2 chương, mỗi chương 5 tiểu mục (1.1–1.5 và 2.1–2.5).** Các danh mục bên dưới là yêu cầu triển khai/biểu đồ/bằng chứng để viết báo cáo chính thức. Chương 1 trình bày kiến thức đã có và trích dẫn; Chương 2 trình bày **thiết kế, cách làm và kết quả riêng của Nhóm 18**. Không trộn giả thuyết với số liệu đã đo.

CHƯƠNG 1. CƠ SỞ LÝ THUYẾT

1.1. Tổng quan kiến trúc mạng Data Center Spine–Leaf

Mục tiêu: giải thích vì sao chọn Spine–Leaf để khảo sát nhiều đường giữa client/server.

Nội dung cần có: khái niệm leaf/spine; east–west traffic; đường đi và số hop; mô hình 3-tier để đối chiếu; kết nối nhiều đường; oversubscription và điều kiện để đạt non-blocking (không khẳng định mọi Spine–Leaf đều non-blocking); giới hạn khi dùng L2/loop prevention.

Minh chứng cần vẽ: hình kiến trúc Spine–Leaf 2×2 và bảng điểm khác biệt với traditional 3-tier (ở mức thiết kế, không phải số liệu đo của nhóm).

Nguồn cần tra cứu: giáo trình mạng, tài liệu kỹ thuật Data Center/fabric có nguồn và ngày truy cập; ghi rõ trích dẫn trong báo cáo.

1.2. Đặc thù lưu lượng AI Inference trong mạng Data Center

Mục tiêu: giải thích workload vì sao liên quan đến bài toán mạng mà **không** phải triển khai LLM thật.

Nội dung: giao tiếp client–server request/response; tải đồng thời, request rate, payload, service time; TCP/HTTP ở tầng ứng dụng/giao vận; phân biệt latency mạng và latency xử lý AI. Không lấy hiện tượng truyền đồng bộ của cụm huấn luyện phân tán làm mục tiêu chính.

Minh chứng: sơ đồ 1 request–response; bảng các tham số workload của emulator; nêu rõ profile mô phỏng không phải trace AI production.

1.3. Bài toán nghẽn mạng và hiện tượng điểm nghẽn (Bottleneck)

Nội dung: capacity/bandwidth, throughput, link utilization, RTT, transmission delay `d_trans = L/R`, queueing delay, packet loss, ảnh hưởng số luồng đồng thời; bão hòa và tranh chấp tài nguyên mạng. Phân biệt **độ trễ truyền** `L/R`, **RTT** đo hai chiều và **thời gian ứng dụng**.

Minh chứng: sơ đồ hàng đợi liên kết; bảng định nghĩa metric + công cụ/đơn vị; ví dụ nguyên lý, không dựng số liệu thực nghiệm của nhóm.

1.4. Công nghệ giám sát và hỗ trợ điều chỉnh cấu hình mạng

Nội dung: cách thu chỉ số interface/port counter, ping/iperf3/tcpdump; phát hiện theo luật/ngưỡng và baseline; thiết kế cảnh báo; báo cáo before/after; cơ chế human-in-the-loop. Phân biệt hỗ trợ thử cấu hình với điều khiển SDN tự động, anomaly detection và self-healing.

Minh chứng: sơ đồ luồng `Monitor → Detect → Recommend → Confirm → Apply → Verify → Compare` và bảng tiêu chí theo dõi.

1.5. Môi trường mô phỏng và công cụ triển khai

Nội dung: Mininet, Open vSwitch, Linux networking, Python, cơ chế emulation/namespace, vai trò của Ubuntu VM trên Windows/macOS, khác biệt VM x86/ARM, công cụ đo, rủi ro timing/counter trên VM. Nêu mục tiêu tái lập môi trường bằng scripts và versions.

Minh chứng: sơ đồ host OS → Ubuntu VM → testbed; bảng stack và trách nhiệm từng thành phần.

CHƯƠNG 2. THIẾT KẾ HỆ THỐNG, TRIỂN KHAI VÀ THỰC NGHIỆM ĐÁNH GIÁ

2.1. Bối cảnh, hiện trạng bài toán và câu hỏi nghiên cứu

Bối cảnh: doanh nghiệp **giả định** có dịch vụ AI inference; Network/System Engineer cần đánh giá cấu hình khi tải tăng trước khi triển khai. Không tự nhận đã khảo sát doanh nghiệp cụ thể.

Hạn chế hiện tại: thiếu môi trường kiểm thử tái lập; benchmark rời rạc không cho quyết định cấu hình; thử trên hạ tầng thật có rủi ro và tốn chi phí; latency ứng dụng phụ thuộc GPU làm nhiễu phép đo nếu dùng LLM thật.

Phải ghi thành mục con: câu hỏi RQ chính và RQ1–RQ3 như phần A.2; mục tiêu (testbed + đo + detect + recommend + human-approved apply + compare); tiêu chí thành công.

Đầu ra viết báo cáo: bảng `Vấn đề → Hệ quả → Chức năng xử lý → Bằng chứng kiểm thử`; không lặp lại toàn bộ Chương 1.

2.2. Phân tích yêu cầu, đầu vào/đầu ra và phạm vi nguyên mẫu

Functional requirements:

ID	Yêu cầu chức năng	Điều kiện hoàn thành
F01	Chạy topology 10 node và 2 đường spine	ping/route/interface chứng minh kết nối
F02	Sinh traffic request/response theo profile 1/2/4/8 flow	log yêu cầu/phản hồi theo từng test

ID	Yêu cầu chức năng	Điều kiện hoàn thành
F03	Thu metric theo flow/link	RTT, throughput, utilization có timestamp và unit
F04	Chẩn đoán/cảnh báo bottleneck	rule xác định link, so khớp port counter và baseline
F05	Đưa khuyến nghị cấu hình có điều kiện	mỗi khuyến nghị có lý do; không in khuyến nghị nếu không đủ bằng chứng
F06	Cho phép Apply/Ignore có xác nhận	chỉ apply preset trên testbed; log action, kiểm tra trạng thái sau Apply
F07	Verify và Compare A/B	chạy cùng offered load, bảng & biểu đồ trước–sau
F08	Lặp, lưu và xuất kết quả	>=3 lần/testcase; raw logs và README tái lập

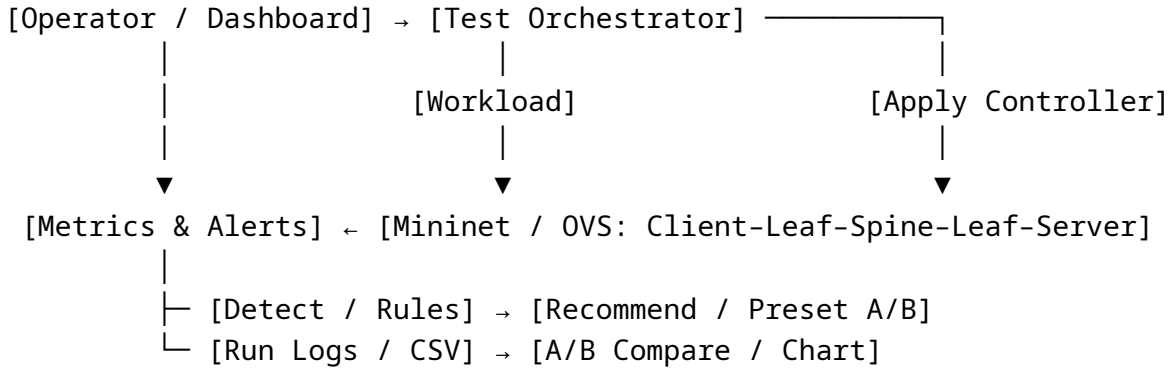
Non-functional: miễn phí; ổn định trong môi trường VM thống nhất; quan sát được; không can thiệp production; cấu hình và số liệu có version/trace; xử lý lỗi Apply rõ ràng. Bảo mật ở mức kiểm thử cục bộ, không xây IAM/PKI.

Đầu vào/đầu ra: dùng bảng phần A.4 và chuyển thành bảng schema (tên trường/đơn vị/default/range/ý nghĩa) khi triển khai. Tách *input kỹ thuật* khỏi *tương tác người vận hành*. Nêu 10 node, 4 endpoint, 1 operator; 8 flow không đồng nghĩa 8 client. Xác định **P0/P1** ngay trong tài liệu.

Đầu ra viết báo cáo: use-case, yêu cầu chức năng, sơ đồ data flow, bảng I/O, giới hạn rõ.

2.3. Thiết kế kiến trúc hệ thống và module chức năng

Kiến trúc bắt buộc:



Phân hệ và giao diện: topology/network bootstrap; workload generator; metrics collector; detection; rule recommendation; confirmation and Apply; verification & comparison; dashboard/CLI; storage/export; logging. Ghi rõ API/CLI command/format dữ liệu trao đổi giữa các module, ai sở hữu từng thành phần.

Topology: 2 client → gateway/router → firewall → Leaf 1 → Spine 1 hoặc Spine 2 → Leaf 2 → 2 AI server; phân tích đường đi và vai trò port; chứng minh không gây loop L2. *Hình thiết kế phải nối đúng từng cạnh*, không dựa vào sơ đồ text lỏng lẻo.

Protocol: tách protocol stack `HTTP/custom app → TCP → IPv4 → Ethernet` và network path; giải thích ARP, ICMP, encapsulation/forwarding/de-encapsulation. Một request có trace/wire capture minh họa.

Apply: preset A/B và điều kiện kích hoạt; chỉ đổi testbed khi xác nhận; ghi old/new config; kiểm tra đường đi sau đổi. Không hứa ECMP nếu chỉ cài static OpenFlow flow split.

Đầu ra viết báo cáo: hình kiến trúc hệ thống; topology; sequence Apply; bảng module/interface; flow/protocol diagram; decision table rule.

2.4. Xây dựng nguyên mẫu và kịch bản thực nghiệm

Triển khai: Ubuntu VM + Mininet/OVS; scripts cài đặt; topology 10 node; server emulator và concurrent client; monitoring collector/alert; dashboard + Apply; runner; raw log exports; cấu hình A và B.

Kịch bản cốt lõi (P0):

Test ID	Profile	Tải (flow)	Mục đích	Đo/kiểm chứng
T01	A (single-path)	1, 2, 4, 8	Baseline tải	RTT, throughput, utilization, bottleneck
T02	B (dual-path có kiểm soát)	1, 2, 4, 8	So sánh với A	đường đi, độ lệch tải, RTT, throughput
T03	A → operator Apply B	mức tải gây nghẽn có kiểm chứng	Demo end-to-end	alert → khuyến nghị → confirm → apply → verify → compare
T04	A/B	ít nhất 3 lần mỗi ô	Tái lập kết quả	raw logs, thống kê trung bình & độ dao động
T05	A (tải thấp)	workload không bão hòa	Kiểm tra detection âm tính	tránh cảnh báo sai

Biến kiểm soát: payload size, request interval, service delay, cấu hình link, khoảng lấy mẫu, thời lượng đo, công cụ, môi trường VM, server CPU background. Với so sánh

A–B: cùng offered load, cùng topology vật lý logic, thay **duy nhất cách chuyển tiếp**; nếu thử tăng capacity link phải tách thành thực nghiệm khác, không gộp vào so sánh A/B.

Rủi ro cần kiểm tra sớm: 8 flow có thể không đủ gây nghẽn; cần hiệu chỉnh bandwidth hoặc offered load trong một giai đoạn pilot rồi **cố định** trước đo chính thức. Các kết luận phải ghi là trên testbed, không đại diện trực tiếp throughput Data Center thật.

Đầu ra viết báo cáo: môi trường, params, testcase, cách chạy, evidence screenshots/logs, commit/config hash, nguyên tắc đo lặp.

2.5. Đánh giá kết quả, thảo luận và kết luận

Bảng/biểu đồ bắt buộc: RTT theo flow; throughput tổng và theo flow; max/per-link utilization; bottleneck/cảnh báo; đối chiếu A–B ở cùng tải; trạng thái Apply và kiểm chứng sau đổi; kết quả lặp lại với độ dao động.

Phương pháp phân tích: nêu công thức metric và đơn vị; minh họa `d_trans = L/R` là kết quả tính lý thuyết; phân biệt RTT ping với response-time ứng dụng; kiểm tra tính hợp lệ của bằng chứng path/counter; tính chênh lệch % sau–trước và khẳng định cải thiện chỉ khi đo được.

Trả lời RQ: kết luận riêng RQ1, RQ2, RQ3; nêu giới hạn VM, giả lập request/response, số node nhỏ, thuật toán khuyến nghị rule-based, kết quả không suy rộng thành hiệu năng mạng production.

Minh chứng: bảng kết quả thực; đồ thị; ví dụ log cảnh báo; ảnh dashboard; kiểm chứng Apply; bảng đối chiếu với tiêu chí nghiệm thu; hướng cải tiến ngoài phạm vi.

C. Phương pháp kiểm thử, nghiệm thu và kiểm soát thay đổi

C.1. Tiêu chí nghiệm thu của prototype (Definition of Done)

- **E01 — Topology:** 10 node logic hiển thị rõ; xác thực IP/interface/route, test được client→server; gateway/firewall không tạo nghẽn ngoài ý muốn.
- **E02 — Đường đi:** chứng minh profile A và profile B thực sự đi khác nhau (counter/trace); không gọi ECMP khi chưa xác minh cơ chế ECMP.
- **E03 — Workload:** Python client/server sinh request/response theo tham số có log, chạy 1/2/4/8 flow từ 2 client logic.
- **E04 — Telemetry:** RTT, throughput, per-link utilization có đơn vị, time window và raw log; d_trans có phép tính minh bạch.
- **E05 — Chẩn đoán:** test quá tải xác định đúng link và test tải thấp không cảnh báo sai; có rule/threshold ghi rõ.
- **E06 — Recommend:** khuyến nghị chỉ dựa trên điều kiện đo được; có lý do và cấu hình thử tương ứng; không tự khẳng định cải thiện trước khi đo lại.

- **E07 — Apply:** chỉ sau operator Confirm, thay đổi một preset cấu hình trong testbed; log before/after, kiểm tra routing; Ignore không thay đổi gì.
- **E08 — Compare:** lặp >=3 lần mỗi kịch bản; bảng A/B cùng tham số, tính % delta, phân tích cả trường hợp không cải thiện.
- **E09 — Tái lập:** checkout source + README + scripts chạy lại trên Ubuntu VM được; số liệu không chỉnh tay.
- **E10 — Báo cáo:** đúng 2 chương; topology/OSI-TCP-IP/metric và AI Networking for AI; có AI Prompts & Verification Log nếu dùng AI code.

C.2. Nguyên tắc cập nhật phạm vi

P0 — bắt buộc: toàn bộ E01–E10; cấu hình 2 profile A/B, dashboard tối thiểu/CLI rõ; một bộ rule detection và recommend; Apply phải thật trong testbed.
P1 — chỉ khi P0 đã ổn định: thêm chế độ thử capacity link độc lập; mở rộng số node/flow; biểu đồ realtime nâng cao; export báo cáo PDF tự động.
Không nhận vào phạm vi: ML dự đoán, AI controller, tự chuyển tuyến khi failure, tự recovery, GPU thật, cloud trả phí, tích hợp thiết bị thật.

Mỗi thay đổi phạm vi cần ghi: **yêu cầu → lý do → ảnh hưởng tuần/người → có phải loại bớt việc P0 hay không → quyết định của nhóm**. Khi rủi ro trễ, cắt P1 trước, không cắt các bằng chứng E01–E10.

C.3. Tài liệu quản lý đi kèm

- `Nhom18_Phan_cong_Checklist_12_tuan.md`: vai trò, lịch 12 tuần, checklist đầu việc, phụ thuộc và nghiệm thu.
- `Nhom18_Progress_Tracker.xlsx`: cập nhật trạng thái từng task, người chịu trách nhiệm, bằng chứng, rủi ro và tổng hợp tiến độ. **Trạng thái mặc định = Chưa làm** cho công việc triển khai (không giả định nhóm đã hoàn thành từ việc có bản đề cương).

C.4. Căn cứ yêu cầu

- File thông báo bài tập lớn môn MI4060 do giảng viên cung cấp: 3 tiêu chí networking (topology; data flow & protocols; quantitative metrics), chọn ít nhất 1 hướng AI; lưu **AI Prompts & Verification Log** khi dùng AI hỗ trợ script.
- Feedback giảng viên trên Teams: tên "Nghiên cứu và xây dựng nguyên mẫu…"; cấu trúc **Chương 1 (1.1–1.5) / Chương 2 (2.1–2.5)**.
- Bản đề cương Nhóm 18 đã thống nhất trước đó: 10 node logic; 2 client/2 server; Mininet/OVS; workload 1/2/4/8; human-approved Apply; no self-healing.

Ghi chú: Các nhận định về việc sử dụng công cụ, rủi ro ECMP/VM và phương pháp kiểm thử là **đề xuất kỹ thuật cần xác minh ở pilot**; không phải kết quả đã được kiểm chứng của nhóm.